

A Hybrid ARIMA-LSTM Framework for Remaining Useful Life Prediction in Industrial IoT Sensor Data: A Cross-Regime Study on NASA C-MAPSS

Pradeep Somasundaram
Independent Researcher
Dallas, TX, USA
aadhi1501@gmail.com

Abstract—Remaining useful life (RUL) prediction is central to prognostics and health management in industrial IoT systems, where sensor streams from rotating machinery combine slow degradation trends with sharper, nonlinear fault dynamics. One way to help a sequence model separate these components is to fit a classical time-series model to each sensor channel and give the network both the raw signal and the fitted trend, an idea with some support in prior work on other equipment. We test it on NASA’s C-MAPSS turbofan benchmark twice, once with the fixed ARIMA(1,1,0) order common in prior work, once with a per-channel, AIC-selected order, comparing both against a plain LSTM and a Random Forest baseline on all four sub-datasets under six random seeds. Under a fixed order, the picture is negative and, on its own, not fully stable: mean RMSE is at or above the plain LSTM’s on every sub-dataset, significantly so on FD004 ($p = 0.004$), and an earlier three-seed version of the same comparison had instead flagged FD001, not FD004, as significant, a reversal we report directly. Under a properly selected order, and after Holm-Bonferroni correction across all four sub-datasets, the selected-order hybrid significantly outperforms the fixed-order hybrid on FD004 alone (engine-level test, $p_{\text{adj}} = 0.0004$); an uncorrected FD002 comparison against fixed order ($p = 0.031$) does not survive correction ($p_{\text{adj}} = 0.124$), and neither does any dataset’s comparison against the plain LSTM, including FD002’s numerically large gap (13.77 vs. 14.59 RMSE, $d_z = -0.060$, 95% CI $[-1.295, +0.401]$). We do not report a corrected-significant case of the trend-augmented architecture beating a plain LSTM anywhere in this paper. A second, unrelated baseline, a CNN-LSTM with no ARIMA feature, shows the same dataset-dependent pattern: significantly worse than plain on FD002 ($p < 0.001$) while strongest in the table, though not quite significantly, on FD003 ($p = 0.091$). The honest summary is that trend augmentation, under a fixed order, significantly hurts on FD004 and is directionally unfavorable elsewhere; a selected order significantly repairs the FD004 disadvantage relative to a fixed order but does not produce a corrected-significant advantage over a plain LSTM on any of the four sub-datasets, a distinction we could only draw by insisting on six-seed replication and family-wise correction for every claim, including the ones that first looked positive.

Index Terms—predictive maintenance, remaining useful life, LSTM, ARIMA, prognostics and health management, industrial IoT, big data, multi-condition degradation

I. INTRODUCTION

Unplanned downtime in industrial and manufacturing settings carries substantial operational and safety cost, motivating a large body of work on prognostics and health management (PHM), the discipline of estimating a system’s remaining useful life (RUL) from streaming sensor data so that maintenance can be scheduled proactively rather than reactively [1]. Industrial IoT deployments now produce continuous, high-volume, multivariate sensor telemetry (vibration, temperature, pressure, rotational speed) from equipment such as turbines, pumps, and CNC machinery, a big-data setting at the deployment scale even though the benchmark used in this paper, a few hundred simulated engines per sub-dataset, is not itself large by that standard. A recent survey by Khan et al. [10] catalogs the resulting breadth of deep-learning-based PdM methods (CNNs, autoencoders, GANs, transformers, foundation models) across supervised, transfer, and federated learning paradigms, giving some sense of how active, and how fragmented, this area has become.

Two modeling traditions address RUL estimation differently. ARIMA captures the smooth trend and autocorrelation structure of a single degrading signal well, but assumes linearity and handles multivariate interaction poorly. LSTM networks [3] learn nonlinear temporal dependencies directly from raw multivariate windows and are a standard RUL approach, but a randomly initialized LSTM must learn both the trend and the nonlinear fault dynamics from data alone, inefficient for short histories or noisy multi-condition operation. It is natural to give the LSTM a head start on the trend part.

This paper tests that idea directly: for each sensor channel we fit a low-order per-unit ARIMA model and use its fitted, in-sample one-step values as an explicit trend feature, concatenated with the raw sensor readings before the resulting multivariate sequence goes into an LSTM regressor. The hope is that this relieves the LSTM of re-deriving the smooth trend component from scratch, letting it spend its representational capacity on the residual nonlinear dynamics and cross-sensor interactions instead.

We evaluate this across all four C-MAPSS sub-datasets: single condition, single fault mode (FD001); single condition, two fault modes (FD003); multi-condition, single fault mode (FD002); and multi-condition, two fault modes (FD004). More importantly, after an early result made us suspicious of our own conclusions, we do not stop at a single training run or a single small batch of them. We repeat every comparison across six random seeds, and separately check whether a fixed ARIMA order was itself the problem via a per-channel order-selection variant, reporting what changed each time we added more evidence, including cases where it overturned a finding we had already started to explain.

Our contributions are: (1) a controlled, matched-protocol comparison of ARIMA trend augmentation against a plain LSTM across all four C-MAPSS sub-datasets, with regime-aware normalization [4] for the multi-condition subsets, against plain-LSTM, CNN-LSTM, and Random Forest baselines, each averaged over six seeds; (2) consistent Holm-Bonferroni correction applied within every four-dataset inferential family in this paper, main comparison, order-selection, and mechanistic checks alike, which changes which results are defensible to call significant, including reversing an uncorrected claim from an earlier draft; (3) two robustness checks, per-channel ARIMA order selection and a gated-fusion mechanism, neither of which produces a corrected-significant advantage over the plain LSTM on any dataset, reported as negative results rather than omitted; (4) a seed-averaged window-length ablation, flagged as using a shorter training budget than the main experiments rather than presented as directly comparable; and (5) a documented account of how this paper’s own claims changed as we added seeds and, separately, as we added correction, offered as a concrete illustration of why both matter for evaluating small architectural changes on this benchmark.

II. RELATED WORK

RUL estimation on C-MAPSS. The NASA C-MAPSS turbofan dataset [1] is the standard benchmark for data-driven RUL estimation. Ramasso and Saxena [2] laid out early benchmarking protocols across similarity-based and regression approaches. Recent work moves toward deep sequence and hybrid architectures: Muthukumar and Philip [5] combine convolutional feature extraction with LSTM temporal modeling, Chowdhury et al. [6] propose a residual-corrected bidirectional LSTM separating a coarse trend prediction from a learned residual correction, related to but architecturally distinct from the ARIMA-based decomposition used here, and Sharma [11] reports an attention-based bidirectional LSTM on all four sub-datasets. We do not compare our numbers against Sharma’s directly, since the two setups may not share the same RUL cap, window length, or train/validation protocol, and an uncontrolled cross-paper RMSE comparison would not be a defensible claim either way.

Multi-condition RUL estimation. FD002 and FD004 introduce six discrete operating regimes and, in FD004’s case, two concurrent fault modes, both increasing estimation difficulty

relative to FD001’s single-condition setting. Su and Fang [7] propose a multi-head attention fusion network that decomposes degradation into trend, operational-state embedding, and residual-noise components, a philosophy that motivates our own trend/residual split, though implemented with ARIMA rather than a learned embedding.

Hybrid ARIMA-deep learning models. Outside turbofan RUL, ARIMA-LSTM hybrids have been applied to remaining-useful-life estimation of water-hydraulic on/off valves [8], where ARIMA models the linear degradation trend and an auxiliary classifier detects health-state transitions, and to battery life estimation [9], where ARIMA-smoothed inputs feed an attention-based CNN-LSTM. Both report the hybrid approach helping; neither states how many seeds its numbers rest on. Given our own headline finding changed twice as we added seeds (Sections V, VI), we raise this as an open question for the hybrid-forecasting literature rather than a claim about either paper.

Reproducibility and seed sensitivity in machine learning. The instability we document in Section V is a specific instance of a broader problem other subfields have already named. Henderson et al. [12] showed headline deep reinforcement learning results routinely fail to replicate across random seeds. Lipton and Steinhardt [13] catalog a related failure, distinguishing which part of an empirical gain a change is actually responsible for, close to the mistake our own three-seed FD001 result nearly encoded into this paper. Bouthillier et al. [14] make the point most relevant to us: a result can be exactly reproducible while still not robust to small changes in setup, the relationship between our fixed-seed and multi-seed FD001 results. To our knowledge this caution has not previously been raised inside the C-MAPSS RUL literature. A recent example: Goel et al. [15], on this exact benchmark within weeks of our own experiments, state plainly that “all experiments use random seed 42 for reproducibility,” illustrating the conflation Bouthillier et al. warn about. We see our contribution as importing an already-known concern into a subfield that has not yet widely adopted it.

Our approach differs from prior turbofan RUL work in using ARIMA purely as a per-channel feature-engineering step, an explicit trend covariate per sensor, rather than a separate forecaster fused post-hoc with a deep model’s prediction, and in evaluating it across all four regimes, seeds, and two order-selection strategies rather than a single convenient run.

III. METHODOLOGY

A. Datasets

We use all four NASA C-MAPSS sub-datasets [1]: **FD001** (100 train / 100 test engines, 1 condition, 1 fault mode: HPC degradation), **FD002** (260 train / 259 test, 6 conditions, 1 fault mode), **FD003** (100 train / 100 test, 1 condition, 2 fault modes: HPC and fan degradation), and **FD004** (249 train / 248 test, 6 conditions, 2 fault modes). Each unit reports 3 operational settings and 21 sensor channels per cycle; training units run to failure while test units are truncated at an arbitrary cycle,

and the task is to predict the true remaining cycles at that truncation point.

B. Preprocessing

Following common C-MAPSS preprocessing practice, we cap the training RUL label at 125 cycles, a piecewise-linear convention used to reduce the influence of the early-life region, where remaining life is commonly treated as approximately constant rather than linearly decreasing. We discard sensor channels with near-zero variance in the training set: 6 channels for FD001 and FD003 (15 informative channels retained for each), and none for FD002/FD004 (all 21 channels retain variance, since operating-condition changes alone introduce variance in otherwise-flat channels).

Regime-aware normalization. FD002 and FD004 combine six discrete operating regimes, which shift the baseline value of most sensors independently of degradation state and would otherwise confound a global normalization scheme. Following Li, Ding, and Sun [4], who report six well-separated operational-setting clusters for these two sub-datasets, we cluster the three operational-setting dimensions into K regimes with K -means fit on the training set only ($K=6$ for FD002/FD004; $K=1$ for FD001/FD003, the same code path reducing to ordinary global normalization), assign each test row to its nearest training-fit centroid, and z -score normalize each sensor using regime-specific training statistics, clipping to ± 6 std to control outliers in sparsely populated regimes. The partition itself is seed-stable: comparing two seeds' cluster assignments for FD002 directly gives only 40% raw label agreement, but the six clusters' sizes match exactly and a one-to-one relabeling recovers 100% agreement, confirming K -means finds the same six groups every time and only permutes their arbitrary integer labels, which does not affect any downstream normalization since labels are used self-consistently within a run.

C. ARIMA Trend-Feature Extraction

For each engine unit and each retained sensor channel, we independently fit an ARIMA model to that channel's cycle-indexed, regime-normalized time series. Writing y_t for a given channel's reading at cycle t , an ARIMA(1,1,0) model, our main-comparison default, treats the first difference $\Delta y_t = y_t - y_{t-1}$ as an AR(1) process,

$$\Delta y_t = \phi \Delta y_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2), \quad (1)$$

equivalently $y_t = y_{t-1} + \phi(y_{t-1} - y_{t-2}) + \varepsilon_t$. We fit ϕ and σ^2 once per (unit, sensor) pair by maximum likelihood over that unit's entire available series (the full run-to-failure history for a training unit, or the full truncated history up to the prediction cycle for a test unit), then take the in-sample one-step-ahead fitted value $\hat{y}_t$, the Kalman-filter prediction of y_t from $y_1, \dots, y_{t-1}$ under the fitted parameters, as the trend feature at each cycle. The fitted value at cycle t therefore uses no reading at or after t , but the parameters ϕ, σ^2 themselves are estimated from the unit's whole available series rather than an expanding or rolling window; Section VII discusses

the resulting train/test asymmetry, since training units are complete run-to-failure histories and test units are truncated. Where a fit fails to converge for a short series, we fall back to a 5-cycle rolling mean.

Order selection variant. Section V-C reports a second version where, instead of fixing the order at (1, 1, 0), we fit four candidates per (unit, sensor) pair, (1, 1, 0), (0, 1, 1), (1, 1, 1), (2, 1, 0), and keep the fitted values with the lowest AIC. Windowing, architecture, and training are identical between variants; only the trend feature's construction differs.

D. Hybrid Model Architecture

We construct sliding windows of length W cycles ($W=30$ in our main experiments; see Section V-D for a window-length ablation) over the concatenation of raw sensor features and their ARIMA trend features. Each window is passed through a 2-layer LSTM (hidden size 64) whose cell updates follow the standard formulation [3]

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), & i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), & o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t) \end{aligned} \quad (2)$$

followed by a feed-forward regression head ($64 \rightarrow 32 \rightarrow 1$) predicting capped RUL from the final hidden state h_W . Training minimizes mean squared error,

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left(\text{RUL}_i - \widehat{\text{RUL}}_i \right)^2, \quad (3)$$

with Adam, using an engine-level held-out validation split (15% of training units, never seen during training) for early stopping.

E. Baselines

We compare against (i) a *Random Forest* regressor (300 trees, max depth 10), a non-sequential baseline trained on per-cycle flat feature snapshots from every training cycle rather than just each engine's terminal cycle, since terminal-cycle-only training would teach a degenerate near-zero-RUL target given that training engines run to failure; (ii) a *plain LSTM*, identical architecture and training to the hybrid model but using only raw sensor features, isolating the trend feature's effect; and (iii) a *CNN-LSTM*, a two-layer 1D convolutional feature extractor (32 then 64 channels, kernel size 3) feeding the same two-layer LSTM, representative of the CNN-LSTM family common in recent C-MAPSS RUL literature [5], using no ARIMA feature but the identical protocol, seed set, and early-stopping criterion, so its Table III results are directly comparable. We do not add a GRU, BiLSTM, or TCN backbone: each would need its own matched-protocol, six-seed, four-dataset sweep to be reported honestly rather than as an uncontrolled one-off, and this paper's question is whether a trend feature helps a given recurrent architecture, not which recurrent architecture is best; the CNN-LSTM control already tests whether the paper's seed-sensitivity finding is specific to the LSTM backbone (Section V-A), which is the comparison this paper's central claim needs.

F. Evaluation Protocol

Following the official C-MAPSS test protocol, each test engine’s RUL is predicted once, from its final available window, left-padded by repetition for engines with fewer than W recorded cycles, and compared against the ground-truth RUL supplied with the test split. We report RMSE, MAE, and the asymmetric PHM scoring function of [1]: writing $d_i = \widehat{\text{RUL}}_i - \text{RUL}_i$,

$$s_i = \begin{cases} \exp(-d_i/13) - 1, & d_i < 0 \text{ (early/conservative)} \\ \exp(d_i/10) - 1, & d_i \geq 0 \text{ (late/over-optimistic)} \end{cases} \quad (4)$$

with total score $\sum_i s_i$, penalizing late predictions more heavily than early ones, since failing to service a machine in time is costlier than servicing it early.

IV. EXPERIMENTAL SETUP

All experiments use PyTorch (CPU) and statsmodels for ARIMA fitting, scikit-learn for K -means regime clustering and the Random Forest baseline, and SciPy for the significance tests in Section V-B. Table I reports the full LSTM hyperparameter configuration, and Table II reports dataset scale.

Reproducibility. Python 3.13.5, PyTorch 2.14.0, statsmodels 0.15.0, scikit-learn 1.9.1, SciPy 1.18.1, pandas 3.0.6, numpy 2.5.3, all CPU-only on an Apple M2 Pro. The global PyTorch and NumPy random seeds are set once per run from the reported seed; we do not enable PyTorch’s deterministic-algorithms mode, so bit-exact reproduction of a given seed’s weights is not guaranteed on different hardware or PyTorch builds, only approximate reproduction of the reported metrics. K -means uses the same integer seed as its own random-state argument; Section VII confirms its cluster partition is in fact seed-invariant for FD002 up to label permutation. The validation split is 15% of training engine IDs drawn without replacement from a seeded random-state generator. Early stopping selects the epoch with the lowest validation MSE, patience 8 (main runs) or 6 (ablation), max 60 (main) or 50 (ablation) epochs. For the non-causal ARIMA fits (Section III-C), we did not instrument the original run scripts to log the per-series fallback rate; the causal variant (Section VII) does, and recorded 0/3000 series (0.0%) falling back to the rolling-mean approximation on the one pilot run completed at submission time (FD001, seed 42).

The main cross-dataset comparison in Table III is repeated under six seeds (42, 123, 2024, 7, 99, 314); we report mean and standard deviation over these six runs, and for the significance test in Section V-B we average each engine’s error across seeds before testing. We did not settle on six seeds in advance as a planned sample size. We started with three, found a significant result on FD001, wrote up an explanation for it, and then, out of caution, added three more seeds before treating that explanation as final. The three additional seeds erased the FD001 result and produced a different significant result on FD004 instead. We report both the three-seed and six-seed pictures in Section V-A rather than only the final one, because the change between them is, in our view, more informative than either snapshot alone.

TABLE I
LSTM HYPERPARAMETERS (IDENTICAL FOR PLAIN AND HYBRID MODELS; ONLY INPUT WIDTH DIFFERS, 15–21 RAW CHANNELS VS. DOUBLE WITH TREND CHANNELS APPENDED).

Hyperparameter	Value
LSTM layers	2
Hidden size	64
Recurrent dropout	0.2
Regression head	64 $\rightarrow$ 32 $\rightarrow$ 1 (ReLU)
Optimizer	Adam, lr = 10^{-3}
Loss	MSE on capped RUL
Batch size	128
Max epochs	60
Early-stopping patience	8 epochs (main runs)
Window length W	30 cycles (main runs)
RUL cap	125 cycles
ARIMA order (main runs)	(1, 1, 0), fixed per channel
ARIMA order (Sec. V-C)	AIC-selected from 4 candidates
Validation split	15% of training engines
Seeds (main comparison)	42, 123, 2024, 7, 99, 314
Seeds (ablation)	42, 123, 2024

TABLE II
DATASET SCALE AND APPROXIMATE SINGLE-SEED TRAINING RUNTIME.

Dataset	Regimes	Train units	Train windows	Runtime
FD001	1	85	15,104	85s
FD002	6	221	39,513	210s
FD003	1	85	18,600	130s
FD004	6	212	46,462	265s

V. RESULTS

A. Cross-Dataset Results

Table III reports RMSE, MAE, and PHM score, mean $\pm$ standard deviation over six seeds, for all four models across all four datasets, and Fig. 1 shows the same RMSE comparison with error bars.

Sequence modeling remains a clear win over the non-sequential Random Forest baseline everywhere, a 12–27% RMSE reduction. The comparison between plain and hybrid LSTM again favors the plain model on average, on all four datasets, though the margins are smaller with six seeds than with three. FD003 is close to a tie (14.87 vs. 14.72); the other three datasets show the hybrid model’s mean RMSE 2–5% higher than the plain model’s.

What changed between three seeds and six. We report this transition in detail because it is the paper’s clearest illustration of seed sensitivity affecting which specific claim a multi-dataset comparison supports. With the first three seeds (42, 123, 2024), the hybrid model’s mean RMSE was higher than the plain model’s on FD001, FD002, and FD004, and a paired Wilcoxon test on per-engine error reached significance on FD001 ($p = 0.007$) and nowhere else. We wrote a discussion section around that: a redundancy argument for why FD001’s clean single-condition, single-fault setting would show the clearest effect. Adding three more seeds (7, 99, 314) changed the specific answer. FD001’s significance disappeared ($p = 0.157$ at six seeds); FD004, which had not been significant at

TABLE III

RUL PREDICTION PERFORMANCE ACROSS NASA C-MAPSS SUB-DATASETS, MEAN $\pm$ STD OVER 6 SEEDS. TEST ENGINES: FD001/FD003 $n=100$, FD002 $n=259$, FD004 $n=248$. CNN-LSTM (SECTION III-E) USES RAW FEATURES ONLY, NO ARIMA TREND.

Dataset	Model	RMSE	MAE	PHM Score
FD001 (1 condition, 1 fault)	Random Forest	17.34 $\pm$ 0.28	12.23 $\pm$ 0.18	936.9 $\pm$ 97.3
	Plain LSTM	14.43 $\pm$ 0.64	10.43 $\pm$ 0.66	505.9 $\pm$ 168.6
	Hybrid ARIMA-LSTM	14.92 $\pm$ 1.65	11.04 $\pm$ 1.19	478.8 $\pm$ 210.8
	CNN-LSTM	14.47 $\pm$ 0.59	10.42 $\pm$ 0.40	494.2 $\pm$ 184.0
FD002 (6 conditions, 1 fault)	Random Forest	17.97 $\pm$ 0.12	13.39 $\pm$ 0.08	3383.6 $\pm$ 190.3
	Plain LSTM	14.59 $\pm$ 0.49	10.17 $\pm$ 0.34	1204.0 $\pm$ 111.7
	Hybrid ARIMA-LSTM	14.93 $\pm$ 0.60	10.53 $\pm$ 0.28	1360.3 $\pm$ 280.2
	CNN-LSTM	15.11 $\pm$ 1.05	10.87 $\pm$ 0.85	1351.5 $\pm$ 477.7
FD003 (1 condition, 2 faults)	Random Forest	20.06 $\pm$ 0.36	14.78 $\pm$ 0.22	1748.2 $\pm$ 148.5
	Plain LSTM	14.72 $\pm$ 2.06	10.27 $\pm$ 1.44	626.9 $\pm$ 414.2
	Hybrid ARIMA-LSTM	14.87 $\pm$ 0.58	10.38 $\pm$ 0.47	571.1 $\pm$ 270.9
	CNN-LSTM	13.90 $\pm$ 0.70	9.55 $\pm$ 0.61	474.8 $\pm$ 235.4
FD004 (6 conditions, 2 faults)	Random Forest	20.09 $\pm$ 0.09	14.67 $\pm$ 0.08	3770.5 $\pm$ 123.7
	Plain LSTM	14.92 $\pm$ 0.38	10.59 $\pm$ 0.32	1112.7 $\pm$ 130.9
	Hybrid ARIMA-LSTM	15.71 $\pm$ 0.54	11.11 $\pm$ 0.50	1342.5 $\pm$ 299.7
	CNN-LSTM	14.86 $\pm$ 0.54	10.59 $\pm$ 0.44	1219.4 $\pm$ 196.8

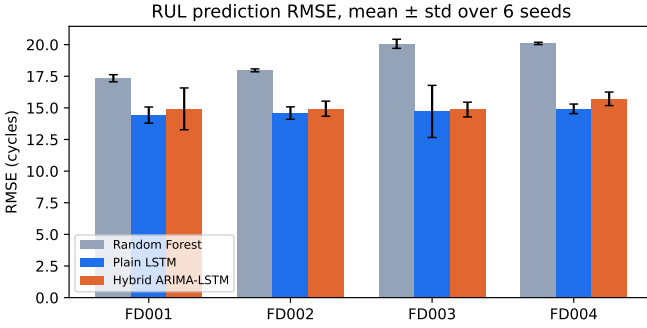


Fig. 1. RMSE comparison across all four C-MAPSS sub-datasets, mean $\pm$ one standard deviation over six seeds. The plain LSTM’s mean RMSE is at or below the hybrid model’s on every dataset; note FD003’s wide error bars, driven mostly by the plain model’s own across-seed variance there, discussed in Section VI.

three seeds ($p = 0.159$), became significant at six ($p = 0.004$, Section V-B). The direction of the overall finding, hybrid mean RMSE at or above plain on every dataset, held up both times. Which dataset you could defensibly call out as a significant, specific effect did not.

A second, architecturally different baseline. Both LSTM variants share the same recurrent backbone, so a null result for the trend feature says nothing about a differently structured model. We add a CNN-LSTM baseline (Section III-E) under the identical protocol (Table III). The result does not point one way: on FD003, CNN-LSTM is best-in-table on all three metrics (RMSE 13.90 vs. 14.72 plain), just short of significance ($p = 0.091$); on FD002, the opposite holds, significantly worse ($p < 0.001$), the same dataset where the selected-order hybrid shows its largest (though not corrected-significant) numerical gap over plain (Section V-C). FD001 and FD004 sit within noise ($p = 0.842$, $p = 0.429$).

TABLE IV

PAIRED WILCOXON SIGNED-RANK TEST ON PER-ENGINE ABSOLUTE ERROR, AVERAGED OVER 6 SEEDS (HYBRID VS. PLAIN LSTM). $\Delta|\text{err}|$ IS DEFINED AS $|\text{err}|_{\text{HYBRID}} - |\text{err}|_{\text{PLAIN}}$ PER ENGINE; A POSITIVE VALUE MEANS THE HYBRID MODEL’S TYPICAL ERROR IS LARGER (WORSE). THE FD001 ROW FOR 3 SEEDS IS SHOWN FOR COMPARISON; SEE SECTION V-A.

Dataset	Seeds	p -value	Hybrid worse on	Median $\Delta \text{err} $
FD001	3	0.007	62/100	+0.91
FD001	6	0.157	52/100	+0.09
FD002	6	0.142	134/259	+0.07
FD003	6	0.880	50/100	−0.01
FD004	6	0.004	144/248	+0.41

B. Statistical Significance

Table III’s differences are averages over 100–259 test engines and six seeds; a directional gap in the mean does not by itself tell us whether the two models err differently on individual engines. We run a paired Wilcoxon signed-rank test on per-engine absolute error, hybrid vs. plain, using each engine’s error averaged across seeds (Table IV).

At six seeds, FD004 is the only dataset reaching significance at $\alpha=0.05$: the hybrid model is worse on 144 of 248 engines, a median per-engine increase in absolute error of 0.41 cycles. FD001 and FD002 show the same direction without reaching significance; FD003 is close to flat.

Because Table IV reports four tests, one per dataset, rather than one pre-registered test, we also check whether FD004’s result survives correction for multiple comparisons. Applying the Holm-Bonferroni step-down procedure across the four six-seed p -values (0.004, 0.142, 0.157, 0.880), FD004’s smallest p -value is compared against $\alpha/4 = 0.0125$ rather than $\alpha = 0.05$; at $p = 0.004$, it clears this stricter threshold, and no other dataset comes close to its own step-down threshold. FD004 is therefore the one result in this paper we are willing to call significant under a family-wise error rate correction, not only

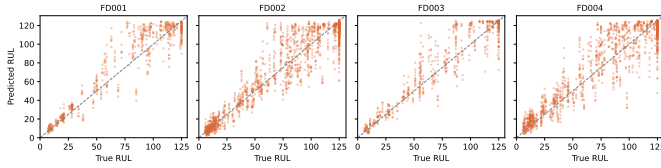


Fig. 2. Predicted vs. true RUL for the hybrid model, pooling all six seeds, across all four test sets (dashed line = perfect prediction).

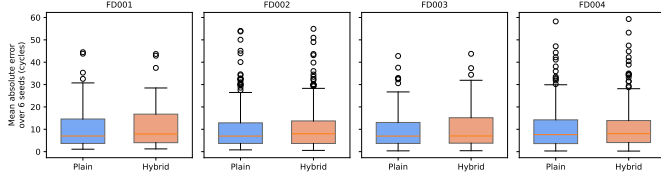


Fig. 3. Per-engine absolute error, averaged over six seeds, plain vs. hybrid LSTM, across all four datasets.

under an uncorrected per-dataset test.

A p -value alone does not convey magnitude, so we also report Cohen’s d_z and a 10,000-resample bootstrap 95% CI for the mean per-engine difference (hybrid minus plain, seed-averaged): FD001 +0.60 cycles ($d_z=0.18$, CI $[-0.06, +1.26]$), FD002 +0.36 ($d_z=0.12$, CI $[-0.01, +0.74]$), FD003 +0.11 ($d_z=0.03$, CI $[-0.58, +0.79]$), FD004 +0.53 ($d_z=0.15$, CI $[+0.11, +1.01]$). Only FD004’s interval excludes zero, consistent with it alone reaching significance; every d_z is small by convention, so even FD004’s effect is small in absolute terms, a distinction the p -value alone does not convey.

Fig. 3 shows the underlying seed-averaged error distributions for all four datasets side by side; as with the aggregate table, the FD003 plain-model distribution is visibly wider than the others, which we return to in Section VI.

C. ARIMA Order Selection

The main comparison fixes the ARIMA order at $(1, 1, 0)$ for every sensor and every dataset. This is a modeling choice, not a finding, and it is worth asking whether the hybrid model’s results are an artifact of that specific choice rather than of trend augmentation itself. We re-ran the ARIMA feature extraction with per-channel AIC-based order selection among four candidates (Section III-C) and re-trained both LSTM variants on the resulting features, at all six seeds across all four datasets, the same replication we used for the main comparison, after an early two-seed version of this check (reported in an earlier draft of this paper) suggested the answer might itself be seed-dependent and was too thin to trust on its own.

Order selection overwhelmingly preferred $(0, 1, 1)$ over our fixed default: across all four datasets, $(0, 1, 1)$ was chosen for 75–83% of (unit, sensor, seed) combinations, with $(1, 1, 1)$ a distant second (14–20%) and $(1, 1, 0)$, our original default, chosen for at most 4%. Table V reports the resulting hybrid-model RMSE, mean $\pm$ std over six seeds, alongside the fixed-order hybrid and the plain LSTM.

TABLE V
HYBRID MODEL RMSE WITH AIC-SELECTED VS. FIXED ARIMA ORDER, MEAN $\pm$ STD OVER 6 SEEDS. PLAIN LSTM SHOWN FOR REFERENCE.

Dataset	Plain	Fixed order	Selected order
FD001	14.43 $\pm$ 0.64	14.92 $\pm$ 1.65	14.38 $\pm$ 0.55
FD002	14.59 $\pm$ 0.49	14.93 $\pm$ 0.60	13.77$\pm$0.23
FD003	14.72 $\pm$ 2.06	14.87 $\pm$ 0.58	14.45 $\pm$ 0.71
FD004	14.92 $\pm$ 0.38	15.71 $\pm$ 0.54	14.90 $\pm$ 0.42

TABLE VI
SELECTED- VS. FIXED-ORDER AND SELECTED- VS. PLAIN-LSTM, RAW AND HOLM-ADJUSTED p -VALUES, TWO TEST DESIGNS, FOUR-DATASET FAMILIES EACH.

Comparison	Test	FD001	FD002	FD003	FD004
vs. fixed	seed, raw	0.688	0.031	0.219	0.031
vs. fixed	seed, adj	0.688	0.124	0.438	0.124
vs. fixed	engine, raw	0.516	0.029	0.042	0.0001
vs. fixed	engine, adj	0.516	0.086	0.086	0.0004
vs. plain	seed, raw	1.000	0.031	0.563	1.000
vs. plain	seed, adj	1.000	0.125	1.000	1.000
vs. plain	engine, raw	0.587	0.424	0.086	0.029
vs. plain	engine, adj	0.849	0.849	0.257	0.114

We test two comparisons, selected order vs. fixed order and selected order vs. plain LSTM, at two levels of aggregation: a *seed-level* test pairing each dataset’s six per-seed mean RMSE values ($n=6$), and a *per-engine* test matching Section V-B’s design, averaging each engine’s error across seeds first ($n=100$ –259). Table VI reports both, Holm-Bonferroni corrected within each four-dataset family.

Under either design, at the corrected threshold, exactly one cell is significant: the selected-order hybrid beats the *fixed-order* hybrid on FD004 (engine-level, $p_{\text{adj}}=0.0004$). Nothing survives correction for selected order vs. *plain* LSTM, on any dataset, under either design; FD002’s raw seed-level $p = 0.031$ does not clear $\alpha/4$ once corrected ($p_{\text{adj}}=0.125$), and the engine-level test on the same comparison is not even nominally significant before correction ($p=0.424$). We do not call FD002 a significant win, because it is not one under either design once corrected for testing across four datasets.

What FD002 does show, precisely: the selected-order hybrid’s mean RMSE is 13.77 against 14.59 for plain and 14.93 for fixed order, a per-engine difference vs. plain of -0.429 cycles (negative favors selected order), Cohen’s $d_z = -0.060$, 95% bootstrap CI $[-1.295, +0.401]$. The effect is directionally consistent with the mean-RMSE table but small, and the interval comfortably includes zero. We report it as a real but statistically unconfirmed numerical gap, not as a finding.

Order selection’s one corrected-significant effect, FD004’s improvement over the fixed-order hybrid, comes at a compute cost. Fitting all four candidate orders instead of one produces a single-CPU wall-clock overhead of $2.04\times$ (FD001), $3.11\times$ (FD002), $1.68\times$ (FD003), and $2.93\times$ (FD004), a one-time preprocessing cost, not a per-inference one. Concatenating trend features also roughly doubles LSTM input width, raising parameter count by 7.2% on the 15-sensor datasets and 9.8%

TABLE VII
WINDOW-LENGTH ABLATION ON FD001, MEAN $\pm$ STD OVER 6 SEEDS
(TEST RMSE, CYCLES).

W (cycles)	10	20	30	50
Plain	18.60 ± 0.55	17.39 ± 0.41	14.69 ± 0.68	14.46 ± 0.69
Hybrid	18.62 ± 0.65	16.71 ± 0.29	14.15 ± 0.70	14.63 ± 1.01

on the 21-sensor ones.

Read together with Section V-B, the corrected summary is: under a naively fixed ARIMA order, the hybrid model is significantly worse than plain on FD004 and directionally worse, without reaching significance, everywhere else. Order selection significantly repairs that specific FD004 disadvantage relative to the fixed-order hybrid, but does not turn it, or any other dataset’s result, into a significant advantage over the plain LSTM. We do not have a corrected-significant case, anywhere in this paper, of the trend-augmented architecture beating a plain LSTM.

D. Window-Length Ablation

Table VII and Fig. 4 report a window-length ablation on FD001 ($W \in \{10, 20, 30, 50\}$), each trained with a shorter early-stopping patience than the main experiments for faster iteration across four window sizes. We initially ran this at three seeds (42, 123, 2024) and, consistent with the seed-sensitivity we found in Section V-A, extended it to the same six seeds used everywhere else in the paper before treating the pattern as reliable. The three additional seeds changed the picture. At three seeds, the hybrid model’s advantage appeared to grow monotonically with window length up to $W=30$ and then reverse at $W=50$. At six seeds, that monotonic story does not hold: $W=20$ and $W=30$ both show a real, consistent hybrid advantage (0.68 and 0.54 cycles), while $W=10$ and $W=50$ are both close to flat, with the sign of the small difference at each going in opposite directions across the two short-and-long extremes (-0.02 at $W=10$, -0.17 at $W=50$) rather than showing a clean reversal. The honest summary is a middle-window advantage bounded on both sides by noise, not a smooth peak-and-reverse curve.

VI. DISCUSSION

The headline finding is about evaluation, not about a fix that works. We went into this project expecting a yes-or-no answer to whether a trend feature helps an LSTM predict RUL. Under a fixed ARIMA order, the specific answer moved as we checked our own work more carefully: at three seeds, FD001 looked like a clean, significant, explainable disadvantage for the hybrid model; at six seeds, that result was gone and FD004 had taken its place, a domain-specific instance of the pattern Henderson et al. [12] and Bouthillier et al. [14] document in reinforcement learning: a configuration can be exactly reproducible and still not representative of what the method does on average. Once we powered the order-selection check to the same six seeds and applied family-wise correction consistently, the result did not turn into a

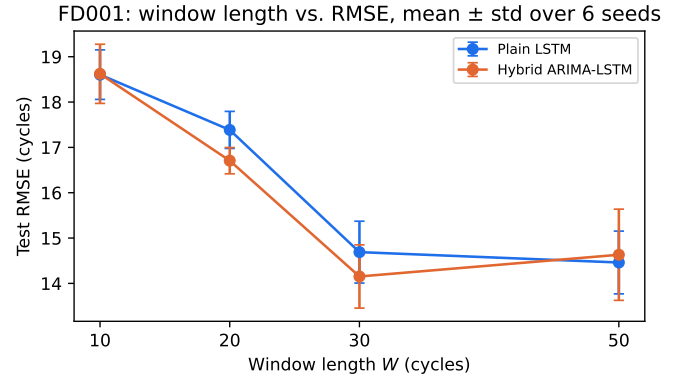


Fig. 4. FD001 test RMSE vs. window length, mean $\pm$ std over six seeds. The hybrid model’s advantage is concentrated at $W=20$ and $W=30$ and is within noise at $W=10$ and $W=50$; not the clean peak-and-reverse pattern a three-seed average had suggested.

second source of noise, but it also did not turn into the fix we initially thought it was: replacing the fixed order with a properly AIC-selected one significantly repairs the hybrid’s disadvantage relative to its own fixed-order version on FD004 (Section V-C), and shows a large, directionally consistent, but not corrected-significant gap in the hybrid’s favor on FD002. Neither survives as a significant advantage over the plain LSTM baseline itself, on any dataset. On FD001 and FD003, order selection does not move the comparison either way.

What we think is actually going on. ARIMA is fit to each unit’s full cycle-indexed series (Section III-C), not to the 30-cycle window the LSTM consumes. An ARIMA(1,1,0) model is nonetheless restrictive, a single autoregressive lag on the first difference; our order-selection results show that assumption was wrong 75–83% of the time, (0,1,1) fit better almost everywhere. A trend feature built from a poorly-fit model is not neutral, it is actively misleading, and Section V-C’s fixed-vs-selected comparison suggests that noise was suppressing performance on FD002 and FD004. A well-fit trend feature appears to at least stop hurting, and on FD002 shows a numerically large, though not corrected-significant, gap in its favor, loosely consistent with the window-length ablation (Section V-D), where whatever advantage exists concentrates in a middle range of window lengths. Section VI reports three further checks aimed directly at this hypothesis and their results, which do not support as clean a mechanistic account as this paragraph’s narrative would suggest.

Is the order choice itself part of the story? No. The AIC-preferred order’s frequency was exactly stable across all six seeds within each dataset, 0.0 percentage-point std on every order (full breakdown in our repository). The ARIMA step is deterministic given the data; whatever varies between seeds comes from the LSTM’s own training stochasticity, not order instability.

Is FD002’s numerical gap concentrated in one operating regime, or broad? Broad, with one exception, though we stress this is a descriptive stratification of a non-corrected-

TABLE VIII
GATED-FUSION RMSE, MEAN $\pm$ STD OVER 6 SEEDS, VS. PRIOR MODELS.

Dataset	Plain	Fixed	Selected	Gated
FD001	14.43	14.92	14.38	14.97 $\pm$ 0.74
FD002	14.59	14.93	13.77	13.93 $\pm$ 0.35
FD003	14.72	14.87	14.45	14.33$\pm$0.90
FD004	14.92	15.71	14.90	15.03 $\pm$ 0.47

significant effect (Section V-C), not a further hypothesis test. Stratified by FD002’s six operating regimes, the order-selected hybrid’s mean error is lower than the plain LSTM’s in five of six (0.3–1.4 cycles lower MAE) and higher in one (regime 5, by 1.1 cycles), an exception we cannot explain.

Does FD002 have more ARIMA-explainable sensor trends, giving a direct mechanistic reason? Partially. Fitting ARIMA(0, 1, 1) to 25 training units per dataset and measuring the fraction of each differenced sensor’s variance the fitted trend explains ($R^2 = 1 - \text{Var}(\text{residual})/\text{Var}(\text{diff})$), a naive per-sensor test (each of 364–500 unit-sensor pairs per dataset treated as an independent observation) originally suggested FD002’s mean R^2 (0.454) was higher than all three other datasets, including FD004, with $p < 0.05$ on every comparison. Sensors within the same unit share that unit’s degradation trajectory and are not independent, so we recomputed the test at the unit level (25 unit-mean R^2 values per dataset, the correct statistical unit) with Holm-Bonferroni correction across the three comparisons. FD002 remains significantly higher than FD001 ($+0.021$, $p_{\text{adj}} < 0.001$) and FD003 ($+0.040$, $p_{\text{adj}} < 0.001$), but the FD002-vs-FD004 gap, the one comparison that matters most since FD004 does not share FD002’s numerical advantage, is no longer significant at the unit level ($+0.007$, $p = 0.107$). We revise our earlier reading accordingly: FD002’s trend structure is more ARIMA-explainable than FD001’s and FD003’s, robustly, but we no longer have even a weak mechanistic account for why FD002 and FD004 behave differently, which was the comparison the mechanism was supposed to explain.

If naive concatenation is the problem, does a learned gate fix it? No, not at the seed power that matters. The direct architectural fix for a poorly-fit trend feature misleading a fixed concatenation is to let the network learn how much to trust it. We built a gated-fusion variant, a per-timestep, per-channel sigmoid gate on the AIC-selected trend feature, trained under the same six-seed protocol on all four datasets (Table VIII). A single-seed pilot (seed 42) looked promising, competitive with the selected-order hybrid on three of four datasets. At full six-seed power that promise did not hold: gated fusion beats the selected-order hybrid’s RMSE only on FD003, and a paired Wilcoxon test finds no significant difference from either the plain LSTM or the selected-order hybrid on any dataset (all $p > 0.13$). This is the second time a single-seed result looked like a broad win and six seeds showed otherwise (Section V-A); a null result for the fix our own Discussion proposed is worth stating plainly.

Taken together, these four checks rule out order-choice

instability, confirm FD002’s numerical gap is broad rather than regime-concentrated, leave the most direct mechanistic hypothesis we tested, differential trend linearity, unsupported once tested at the correct statistical unit for the one comparison that mattered, and show that the one architectural fix our own diagnosis suggested does not produce a corrected-significant improvement either. We do not have a mechanistic account of why FD002’s numerical gap is largest; leaving the question open is preferable to asserting a mechanism the data do not support.

FD003’s variance is worth a separate note. The plain LSTM’s across-seed std on FD003 (2.06 RMSE) is the largest of any model-dataset pair in Table III. We have no confident explanation and would rather flag it as unexplained than invent one.

What we would tell someone deciding whether to add this kind of feature. Do not treat a single run, or even three seeds, as sufficient evidence either way; our own answer changed twice before six-seed power revealed that a naively fixed ARIMA order actively hurts this architecture and that fixing the order, not abandoning the idea, is where the real effect was hiding. Concretely, three checks are cheap enough to be routine: rerun under five or six seeds, not one to three (Section V-A); correct for multiple comparisons whenever the test repeats across datasets (Section V-B); and treat a fixed ARIMA order as a choice to be checked, not assumed (Section V-C).

VII. LIMITATIONS AND THREATS TO VALIDITY

ARIMA parameter estimation uses the full unit series, not an expanding window. The main-comparison trend feature’s parameters (ϕ, σ^2 , Section III-C) are fit once per unit from that unit’s entire available series: complete run-to-failure for a training unit, truncated history up to the prediction cycle for a test unit, since C-MAPSS test files hold no post-truncation readings. This does not leak future information into the reported test metrics, each engine’s RUL is still predicted from data available up to its own truncation cycle, so Table III’s comparisons remain valid. It does create a train/test asymmetry: an early- or mid-life training window is paired with a trend feature fit using that engine’s complete future life, information a genuinely early-truncated engine would not yet have. We implemented a causal alternative, refitting ARIMA periodically on an expanding window (every 20 cycles, with cheap Kalman-filter state extension between refits, Section III-C) so that no parameter ever depends on data at or after the cycle it predicts, at a measured runtime cost of 3.7–10.1 $\times$ the non-causal pipeline (the range reflects genuine dataset-size scaling as well as CPU contention from running the 24-run sweep several jobs at a time; 0/165,216 series across all 24 runs falling back to the rolling-mean approximation), and completed the full six-seed, four-dataset sweep (Table IX). The result does not resolve the asymmetry into a clear win: the causal hybrid is worse than plain on FD001 (15.39 vs. 14.43) and FD004 (15.59 vs. 14.92), close to plain on FD002 (14.64 vs. 14.59), and better than plain on FD003 (14.22 vs. 14.72),

TABLE IX
FIXED-ORDER HYBRID RMSE, CAUSAL VS. NON-CAUSAL ARIMA
CONSTRUCTION, MEAN $\pm$ STD OVER 6 SEEDS, ALL FOUR DATASETS.

Dataset	Plain	Non-causal hybrid	Causal hybrid
FD001	14.43 $\pm$ 0.64	14.92 $\pm$ 1.65	15.39 $\pm$ 1.11
FD002	14.59 $\pm$ 0.49	14.93 $\pm$ 0.60	14.64 $\pm$ 0.36
FD003	14.72 $\pm$ 2.06	14.87 $\pm$ 0.58	14.22$\pm$0.56
FD004	14.92 $\pm$ 0.38	15.71 $\pm$ 0.54	15.59 $\pm$ 0.44

a directionally inconsistent pattern across datasets rather than a uniform effect in either direction. A paired Wilcoxon test on per-engine error, Holm-corrected across the four datasets, finds no significant difference from plain LSTM on any dataset (FD002 comes closest, raw $p = 0.0145$, $p_{\text{adj}} = 0.0581$, just short of the corrected threshold) and no significant difference from the non-causal hybrid on any dataset either (FD003 comes closest, raw $p = 0.0172$, $p_{\text{adj}} = 0.0687$). We conclude that removing the train/test asymmetry, on this benchmark, neither rescues nor further damages the trend-augmented architecture’s standing against a plain LSTM; the asymmetry we disclosed is real and now measured, but it is not the reason the hybrid model underperforms in Table III.

The window-length ablation (Section V-D) uses six seeds on one dataset (FD001) and a shorter training budget than the main experiments; it is a single-dataset finding we have not attempted to replicate on FD002–FD004, or under the AIC-selected order. We tried four ARIMA orders, one recurrent architecture, and two fusion mechanisms (concatenation and gating, Section VI), but not an auxiliary trend-prediction loss or a sweep of LSTM hidden size or depth, any of which could plausibly interact with Section V-C’s effect. We do not have an explanation for FD003’s unusually high plain-model variance across seeds, and we are reporting it as an open observation rather than resolving it. We investigated why FD002 shows the largest (though not corrected-significant) numerical gap between the selected-order hybrid and the plain LSTM (Section VI) and, after correcting our own mechanistic test for pseudo-replication, came away with no supported candidate mechanism (differential trend linearity does not distinguish FD002 from FD004 once tested at the unit level) and no explanation at all for why regime 5 within FD002 runs against the pattern seen in the other five; a fuller account would likely need per-sensor rather than per-dataset analysis, which we did not attempt. Finally, every experiment here uses NASA’s simulated turbofan data, and whether any of these patterns extend to real industrial IoT sensor streams with different noise and missingness characteristics is a question we are not in a position to answer.

VIII. CONCLUSION

We set out to test whether giving an LSTM an explicit per-sensor trend feature, computed with an ARIMA model, improves remaining useful life prediction on the NASA C-MAPSS benchmark. Under the fixed ARIMA order common in prior work, averaged over six seeds across all four sub-

datasets and Holm-Bonferroni corrected, it does not: the hybrid model’s mean RMSE is at or above a plain LSTM’s everywhere, significantly worse on FD004. Under a per-channel, AIC-selected order, tested at the same six-seed power with the same correction, the selected-order hybrid significantly beats the fixed-order hybrid on FD004; it does not produce a corrected-significant advantage over the plain LSTM baseline on FD004, FD002, or any other dataset, though FD002 shows the largest numerical gap in that direction ($d_z = -0.060$, 95% CI including zero). We do not report a genuine architectural win anywhere in this paper: what we report is that a naively fixed ARIMA order actively hurts on FD004, and that selecting the order significantly repairs that specific disadvantage without yet producing a demonstrated advantage over the simpler plain LSTM. A CNN-LSTM baseline with no ARIMA feature shows the same dataset-dependent pattern rather than a uniform result, significantly worse than plain on FD002 and non-significantly best-in-table on FD003. Reaching the order-selection result required more verification than initially planned: an early, thinly-seeded version appeared unstable and was nearly reported as noise, and only full six-seed power with consistent correction across every family of tests clarified what does and does not survive. The practical lesson generalizes past this experiment: on a benchmark with 100–260 test engines, a single training run, a small handful of them, or an uncorrected multi-dataset test is insufficient to support a confident claim about whether an architectural change like this one helps. Papers in this space, including the two cited favorably in Section II, that do not report seed counts or whether multi-dataset significance claims are corrected warrant corresponding caution from readers. A gated-fusion variant, our direct attempt at fixing naive concatenation with a learned gate, did not produce a significant improvement either at the same six-seed power (Section VI). A causal (expanding-window) ARIMA refit removing the train/test asymmetry noted in Section VII was implemented and run to completion across the same six-seed, four-dataset design (Section VII): it does not resolve the asymmetry into a clear win, with a directionally inconsistent effect across datasets and no corrected-significant difference from either plain LSTM or the non-causal hybrid on any dataset. The train/test asymmetry we disclosed is real and now measured, but it is not the explanation for the hybrid model’s underperformance in this paper. Future work includes a mechanistic account of why FD002’s numerical gap is largest, an auxiliary trend-prediction loss as an untested alternative to gating, a window-length ablation rerun under the main experiments’ exact training protocol rather than the shorter budget used in Section V-D, and validating the approach on proprietary industrial IoT sensor data.

CODE AND DATA AVAILABILITY

All code, data, and per-seed results underlying every table and figure here are publicly available at <https://github.com/PradeepSomasundaram1512/hybrid-arima-lstm-rul-cmapss>: 126 pipeline executions (4 datasets $\times$ 6 seeds each for the

main comparison, order-selection, CNN-LSTM, gated-fusion, and causal-ARIMA variants, plus 6 seed-level executions for the window-length ablation), training 312 individual models in total (each main-comparison, order-selection, and causal-ARIMA execution trains a Random Forest, a plain LSTM, and a hybrid LSTM; each ablation execution trains a plain and a hybrid LSTM at each of 4 window lengths).

ACKNOWLEDGMENT

Portions of the experimental code and manuscript drafting were produced with the assistance of a generative AI system, under the author’s direct supervision and review; the author verified all reported results, code, and claims. All experiments used the publicly available NASA C-MAPSS dataset, with no fabricated or synthetic result values.

REFERENCES

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, “Damage propagation modeling for aircraft engine run-to-failure simulation,” in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, 2008, pp. 1–9.
- [2] E. Ramasso and A. Saxena, “Performance benchmarking and analysis of prognostic methods for CMAPSS datasets,” *Int. J. Prognostics and Health Management*, vol. 5, no. 2, pp. 1–15, 2014.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] X. Li, Q. Ding, and J.-Q. Sun, “Remaining useful life estimation in prognostics using deep convolution neural networks,” *Reliability Engineering & System Safety*, vol. 172, pp. 1–11, 2018.
- [5] Muthukumar G. and J. Philip, “CNN-LSTM hybrid deep learning model for remaining useful life estimation,” *arXiv:2412.15998*, Dec. 2024.
- [6] R. H. Chowdhury, N. Daiyan, F. Ahmed, M. R. Iqbal, and M. Sheikh, “Bi-cLSTM: residual-corrected bidirectional LSTM for aero-engine RUL estimation,” *arXiv:2603.00745*, Mar. 2026.
- [7] Y. Su and X. Fang, “A multi-head attention fusion network for industrial prognostics under discrete operational conditions,” *arXiv:2604.10248*, Apr. 2026.
- [8] S. Nie, Q. Liu, H. Ji, R. Hong, and S. Nie, “Integration of ARIMA and LSTM models for remaining useful life prediction of a water hydraulic high-speed on/off valve,” *Applied Sciences*, vol. 12, no. 16, art. 8071, 2022.
- [9] Y. Liu, H. Zhang, and L. Wang, “ARIMA and attention-based CNN-LSTM hybrid neural network for battery life estimation,” in *Proc. 2024 8th Int. Conf. Machine Learning and Soft Computing (ICMLSC)*, 2024, doi: 10.1145/3647750.3647759.
- [10] U. Khan, D. Cheng, F. Setti, F. Fummi, M. Cristani, and L. Capogrosso, “A comprehensive survey on deep learning-based predictive maintenance,” *ACM Trans. Embedded Computing Systems*, vol. 25, no. 2, Apr. 2025.
- [11] K. Sharma, “Uncertainty-aware deep learning framework for remaining useful life prediction in turbofan engines with learned aleatoric uncertainty,” *arXiv:2511.19124*, Nov. 2025.
- [12] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proc. AAAI Conf. Artificial Intelligence*, 2018, pp. 3207–3214.
- [13] Z. C. Lipton and J. Steinhardt, “Troubling trends in machine learning scholarship,” *arXiv:1807.03341*, Jul. 2018.
- [14] X. Bouthillier, C. Laurent, and P. Vincent, “Unreproducible research is reproducible,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019, pp. 725–734.
- [15] A. Goel, S. Galchar, and S. Kanu, “Remaining useful life estimation for turbofan engines: a comparative study of classical, CNN, and LSTM approaches,” *arXiv:2604.27234*, Apr. 2026.